

Student Examination Database

Student Details

Student Name	PRITHVI . S
Register Number	2428C0388
Department	BSc AI&ML
ID	asbrubw2428c0388
College	AKSHAYA COLLEGE OF ARTS AND SCIENCE
Date of Submission	21-9-2026
GIT HUB LINK	https://github.com/prithviai2005/BRU-.git

1. Project Title

Examination Management System Database

2. Problem Statement

Educational institutions need to maintain student examination details, subject information, examination schedules, marks, grades, and final results. Managing these details manually can lead to data duplication, errors, and difficulty in generating student result reports.

The proposed database system stores all examination-related information in structured tables and allows administrators or faculty members to easily enter marks, calculate grades, and generate student result reports.

3. Objectives

1. Store and manage student details.
 2. Maintain subject information.
 3. Store examination records.
 4. Record marks obtained by students.
 5. Automatically calculate grades and results.
 6. Generate individual student result reports.
 7. Reduce data duplication and manual errors.
 8. Provide easy access to examination information.
-

4. Database Design

Database Name

ExaminationDB

Tables Used

Table	Purpose
Students	Stores student information
Subjects	Stores subject information
Examinations	Stores examination details
Marks	Stores marks obtained by students
Results	Stores calculated grade and result

5. Create Database

```
CREATE DATABASE ExaminationDB;
```

```
USE ExaminationDB;
```

6. Student Details Table

```
CREATE TABLE Students (  
    Student_ID INT PRIMARY KEY,  
    Student_Name VARCHAR(100) NOT NULL,  
    Register_No VARCHAR(20) UNIQUE NOT NULL,  
    Department VARCHAR(50),  
    Year INT,  
    Email VARCHAR(100),  
    Phone VARCHAR(15)  
);
```

Sample Data

INSERT INTO Students VALUES

(1, 'Arun Kumar', 'REG001', 'B.Sc AI & ML', 3, 'arun@gmail.com', '9876543210'),
(2, 'Prithvi S', 'REG002', 'B.Sc AI & ML', 3, 'prithvi@gmail.com', '9876543211'),
(3, 'Karthik R', 'REG003', 'B.Sc AI & ML', 3, 'karthik@gmail.com', '9876543212');

7. Subject Details Table

CREATE TABLE Subjects (

Subject_ID INT PRIMARY KEY,
Subject_Code VARCHAR(20) UNIQUE NOT NULL,
Subject_Name VARCHAR(100) NOT NULL,
Max_Marks INT DEFAULT 100

);

Sample Data

INSERT INTO Subjects VALUES

(101, 'AIML301', 'Machine Learning', 100),
(102, 'AIML302', 'Python Programming', 100),
(103, 'AIML303', 'Database Management System', 100),
(104, 'AIML304', 'Artificial Intelligence', 100),
(105, 'AIML305', 'Web Development', 100);

8. Examination Records Table

CREATE TABLE Examinations (

Exam_ID INT PRIMARY KEY,
Exam_Name VARCHAR(100) NOT NULL,
Exam_Type VARCHAR(50),
Exam_Date DATE,
Semester INT

);

Sample Data

INSERT INTO Examinations VALUES

(1, 'Internal Assessment 1', 'Internal', '2026-07-15', 5),
(2, 'Internal Assessment 2', 'Internal', '2026-08-20', 5),
(3, 'Semester Examination', 'End Semester', '2026-09-15', 5);

9. Marks Table

This table connects **students, subjects and examinations**.

CREATE TABLE Marks (

Mark_ID INT PRIMARY KEY,

Student_ID INT,

Subject_ID INT,

Exam_ID INT,

Marks_Obtained INT,

FOREIGN KEY (Student_ID) REFERENCES Students(Student_ID),

FOREIGN KEY (Subject_ID) REFERENCES Subjects(Subject_ID),

FOREIGN KEY (Exam_ID) REFERENCES Examinations(Exam_ID)

);

Sample Data

INSERT INTO Marks VALUES

(1, 1, 101, 3, 85),

(2, 1, 102, 3, 78),

(3, 1, 103, 3, 92),

(4, 1, 104, 3, 74),

(5, 1, 105, 3, 88),

(6, 2, 101, 3, 91),

(7, 2, 102, 3, 86),

(8, 2, 103, 3, 95),
(9, 2, 104, 3, 89),
(10, 2, 105, 3, 93),
(11, 3, 101, 3, 67),
(12, 3, 102, 3, 72),
(13, 3, 103, 3, 64),
(14, 3, 104, 3, 70),
(15, 3, 105, 3, 76);

10. Grade Calculation

We can calculate grades using SQL CASE.

Grade Criteria

Marks	Grade	Result
90–100	A+	Pass
80–89	A	Pass
70–79	B	Pass
60–69	C	Pass
50–59	D	Pass
Below 50	F	Fail

Grade Query

```
SELECT
    s.Student_Name,
    sub.Subject_Name,
    m.Marks_Obtained,

    CASE
        WHEN m.Marks_Obtained >= 90 THEN 'A+'
        WHEN m.Marks_Obtained >= 80 THEN 'A'
```

```

        WHEN m.Marks_Obtained >= 70 THEN 'B'
        WHEN m.Marks_Obtained >= 60 THEN 'C'
        WHEN m.Marks_Obtained >= 50 THEN 'D'
        ELSE 'F'
    END AS Grade,
CASE
    WHEN m.Marks_Obtained >= 50 THEN 'PASS'
    ELSE 'FAIL'
END AS Result
FROM Marks m
JOIN Students s
    ON m.Student_ID = s.Student_ID
JOIN Subjects sub
    ON m.Subject_ID = sub.Subject_ID;

```

11. Student Result Report

The following query generates the complete result of a student.

```

SELECT
    s.Register_No,
    s.Student_Name,
    s.Department,
    sub.Subject_Code,
    sub.Subject_Name,
    m.Marks_Obtained,

CASE
    WHEN m.Marks_Obtained >= 90 THEN 'A+'
    WHEN m.Marks_Obtained >= 80 THEN 'A'
    WHEN m.Marks_Obtained >= 70 THEN 'B'

```

```

        WHEN m.Marks_Obtained >= 60 THEN 'C'

        WHEN m.Marks_Obtained >= 50 THEN 'D'

        ELSE 'F'

    END AS Grade,

    CASE

        WHEN m.Marks_Obtained >= 50 THEN 'PASS'

        ELSE 'FAIL'

    END AS Result

FROM Students s

JOIN Marks m

    ON s.Student_ID = m.Student_ID

JOIN Subjects sub

    ON m.Subject_ID = sub.Subject_ID

WHERE s.Student_ID = 2;

```

Expected Output

Register No	Student	Subject	Marks	Grade	Result
REG002	Prithvi S	Machine Learning	91	A+	PASS
REG002	Prithvi S	Python Programming	86	A	PASS
REG002	Prithvi S	DBMS	95	A+	PASS
REG002	Prithvi S	Artificial Intelligence	89	A	PASS
REG002	Prithvi S	Web Development	93	A+	PASS

12. Total, Average and Overall Result

```
SELECT

    s.Register_No,

    s.Student_Name,

    SUM(m.Marks_Obtained) AS Total_Marks,

    COUNT(m.Subject_ID) AS Total_Subjects,

    ROUND(AVG(m.Marks_Obtained), 2) AS Average_Marks,


CASE

    WHEN MIN(m.Marks_Obtained) >= 50 THEN 'PASS'

    ELSE 'FAIL'

END AS Overall_Result


FROM Students s

JOIN Marks m

    ON s.Student_ID = m.Student_ID

WHERE s.Student_ID = 2

GROUP BY s.Student_ID, s.Register_No, s.Student_Name;
```

Expected Output

Register No	Student	Subject	Marks	Grade	Result
REG002	Prithvi S	Machine Learning	91	A+	PASS
REG002	Prithvi S	Python Programming	86	A	PASS
REG002	Prithvi S	DBMS	95	A+	PASS
REG002	Prithvi S	Artificial Intelligence	89	A	PASS
REG002	Prithvi S	Web Development	93	A+	PASS

13. Database Relationship

The database contains four main relationships:

1. **Students** → **Marks**

- Student_ID connects the **Students** table with the **Marks** table.
- One student can have many marks records.

2. **Subjects** → **Marks**

- Subject_ID connects the **Subjects** table with the **Marks** table.
- One subject can have marks for many students.

3. **Examinations** → **Marks**

- Exam_ID connects the **Examinations** table with the **Marks** table.
- One examination can contain marks for many students and subjects.

4. **Marks as a Linking Table**

- The **Marks** table connects **Students, Subjects, and Examinations**.
- It stores the marks obtained by each student for each subject in a particular examination.

Relationship Diagram

Students

|
| Student_ID



Marks ◀—— Subject_ID —— Subjects



| Exam_ID
|

Examinations

Relationship Type:

- Students → Marks = **One-to-Many**
- Subjects → Marks = **One-to-Many**
- Examinations → Marks = **One-to-Many**

14. Main Features Achieved

- **Student Details** – Stores student name, register number, department, year, email and phone.
- **Subject Details** – Stores subject code, name and maximum marks.
- **Examination Records** – Stores exam name, type, date and semester.
- **Marks Management** – Records marks for every student, subject and examination.
- **Grade Calculation** – Automatically calculates grades using SQL CASE.
- **Result Calculation** – Determines PASS/FAIL based on marks.
- **Student Result Report** – Displays subject-wise marks, grades, total and average.
- **Database Relationships** – Uses primary keys and foreign keys to maintain data integrity.